

Full Length Article

Causality-aware LLM-agent framework for document-level news event extraction with heterogeneous arguments

Bingtao Xu^{id}, Zongye Gu*, Yanling Wang*

School of Journalism and Communication, Tianjin Normal University, Tianjin, 300387, China

ARTICLE INFO

Keywords:

Natural language processing
Document-level event extraction
Causal inference
Agent-based information extraction
Heterogeneous argument roles

ABSTRACT

With the explosion of unstructured text data in news reports, there is an urgent need for automatic event extraction to support timely decision-making. This paper tackles the challenging task of document-level multi-type news event extraction with heterogeneous argument structures. Unlike sentence-level extraction, this task must handle long documents containing multiple events of different types, each with a distinct set of roles (arguments). Existing event extraction approaches struggle with incomplete information and widely scattered clues in lengthy news articles. To address these issues, we propose a causality-aware LLM-agent framework that leverages large language models (LLMs) and an agent-based extraction strategy. First, our method uses an LLM-driven event detector to identify event triggers and types in a document, even when event mentions are sparse or spread across paragraphs. Second, an argument extraction agent performs state-aware role-wise extraction for each event. It initializes a schema-specific role queue and uses QA feedback, including returned spans, confidence scores, and NULL decisions, to adjust subsequent query formulation and local context selection. This yields more complete arguments while keeping predictions grounded in source spans. Third, we incorporate a causal inference module that links events with cause-effect relations, using these relations to infer or validate certain arguments (e.g. shared locations or times) when explicit mentions are missing. We conduct extensive experiments on two public datasets for document-level event extraction from news documents. Results show that our approach achieves state-of-the-art performance, outperforming strong baselines by 3–5% F1. Notably, it maintains high recall on long news documents with multiple events, demonstrating robustness to incomplete and scattered information. These improvements validate the effectiveness of integrating LLM reasoning, agent-based extraction, and causal inference. Our work provides a novel solution for accurate multi-event extraction from complex news documents, and we discuss its significance for downstream news understanding tasks and future extensions to broader domains.

1. Introduction

Every day, massive amounts of news and social media text describe real-world events across diverse domains. Event extraction – identifying events of interest and extracting their key participants and attributes – has become a crucial natural language processing task to transform such unstructured text into structured information. Extracted event knowledge supports a range of intelligent applications, from news aggregation and knowledge graph construction to public security monitoring and financial risk analysis. Classic benchmarks like MUC and ACE (Cunha et al., 2023) drove early progress in event extraction by defining standard event schemas and evaluation datasets. For example, the ACE 2005 program defined several event types (e.g. Attack, Die, Meet) each with specific argument roles (Agent, Victim, Location, Time, etc.). Traditional approaches treated event extraction at the sentence level, using pipeline

architectures: first detect event triggers (words signaling an event occurrence), then classify the event type and extract associated arguments in the same sentence. Early methods for event extraction relied on pattern matching and feature-based machine learning. Ahn (2006) provided a foundational analysis of the stages of event extraction, from trigger detection to role filling. With the rise of statistical NLP, supervised models were developed on ACE data. For instance, Nguyen and Grishman (2015) used convolutional neural networks for event detection, and Nguyen et al. (2016) proposed a joint recurrent neural network model to simultaneously identify triggers and arguments. These neural models improved over prior systems by learning distributed representations of words and capturing context. Subsequent works introduced representations of entity interactions and global features. Hong et al. (2011) leveraged cross-entity inference to improve argument identification. Chen et al. (2015) applied dynamic multi-pooling CNNs to

* Corresponding authors.

E-mail addresses: bingtaoxu@stu.tjnu.edu.cn (B. Xu), zongyegu@stu.tjnu.edu.cn (Z. Gu), yanlingwang.tjnu@outlook.com (Y. Wang).<https://doi.org/10.1016/j.neunet.2026.109629>

Received 30 November 2025; Received in revised form 10 July 2026; Accepted 7 September 2026

Available online 10 September 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

better capture triggers and arguments in text. Graph-based approaches were also explored: [Nguyen and Grishman \(2018\)](#) built argument-aware graph convolutional networks for event detection, and [Liu et al. \(2018\)](#) aggregated information with an attention-based graph to jointly extract multiple events. These works showed the benefit of modeling dependencies among triggers, entities, and relations within a sentence.

However, a limitation of early research is the focus on extracting events within single sentences. Real-world articles often describe complex scenarios involving multiple events, with important details scattered across sentences or paragraphs. To address this, recent studies have shifted toward document-level event extraction (DEE), which aims to extract all event mentions from a document and allow arguments to be filled using cross-sentence context. Pioneering work by [Yang et al. \(2018\)](#) introduced DCFEE, a document-level Chinese financial event extraction system that first identifies key sentences and then aggregates their information for final event records. [Zheng et al. \(2019\)](#) proposed Doc2EDAG, which represents each event as a directed acyclic graph of entity mentions, enabling an end-to-end framework for document-level event extraction. These methods tackled challenges unique to the document level: arguments may be mentioned far from the trigger, and multiple events (even of the same type) can occur in one document. Subsequent approaches improved modeling of long text and multi-event interactions. For example, [Yang et al. \(2021\)](#) developed a parallel prediction network (DE-PPN) to decode multiple events non-autoregressively, and [Xu et al. \(2021\)](#) designed a heterogeneous graph interaction model with a neural “tracker” to capture entity coreference and context for events across sentences. [Liang et al. \(2022\)](#) introduced RAAT, an attention-based transformer that incorporates relations between sentences and events to better aggregate dispersed arguments. Most recently, [Wang et al. \(2023a\)](#) proposed using event proxy nodes and a Hausdorff distance objective to explicitly model multiple event occurrences in a document. These advances have significantly improved document-level event extraction performance.

Despite this progress, important challenges remain. Document-level systems must handle heterogeneous argument structures for different event types. Each event type (e.g. Natural Disaster, Personnel Appointment, Transaction) may require extracting a unique set of roles. Prior work often uses template-based decoders or separate classifiers per role, but designing a unified model that flexibly handles all event schemas is non-trivial. Moreover, real news documents often have missing information: not every role is explicitly mentioned. For example, a report on a protest might mention the location and participants but omit the exact time. Models should robustly extract what is present while not hallucinating missing details. Another challenge is that events in a document can be sparsely distributed – some paragraphs may contain dense event descriptions, while others are mostly background. Models need to distinguish relevant context from irrelevant content and possibly link information scattered in different parts of the text. Recent studies have highlighted the difficulty of capturing long-range dependencies under these conditions. Furthermore, existing DEE approaches largely rely on supervised learning with fixed training data, struggling in cases where an event type was unseen or under-represented in training (the long-tail or zero-shot scenario).

In parallel, the NLP field has seen the emergence of large language models (LLMs) like GPT-3/4 that demonstrate strong abilities in reading comprehension and open-ended question answering. This raises the question: can we leverage LLMs’ broad knowledge and reasoning for event extraction? Some early explorations treat event extraction as a generation or QA task and employ LLMs in a prompt-based manner. For instance, generative approaches such as Text2Event ([Lu et al., 2021](#)) generate a structured sequence for events including arguments, and [Paolini et al. \(2021\)](#) frame IE as translating text to a structured “augmented natural language” format. While these methods tap into pre-trained language knowledge, they often struggle to enforce extraction of all required arguments and to handle multiple events simultaneously. Recent work has attempted to fine-tune or instruction-

tune LLMs for event extraction: for example, [Ma et al. \(2022\)](#) introduced PAIE, using prompt templates to guide a model like BART to fill argument slots; [Hsu et al. \(2021\)](#) proposed DEGREE, a GPT-based model for few-shot event extraction; and [Yu et al. \(2024\)](#) incorporate LLM-generated chain-of-thought explanations to improve extraction (EE-LCE). Research has found that while LLMs can capture semantic clues, direct in-context prompting often yields subpar accuracy compared to task-specific smaller models. This is partly due to the difficulty of prompting an LLM to produce a fully structured output with all arguments in the right format. Our work seeks a middle ground, but the division of labor is deliberate rather than ad hoc. In news documents, trigger mentions are often lexically diverse, context-dependent, and sometimes only weakly signaled by local surface patterns, so we use the LLM at the front end to scan the full document and propose high-recall event candidates. By contrast, argument extraction requires role-specific span grounding under a fixed schema, where direct LLM generation is more prone to hallucinated fillers, imprecise boundaries, and formatting inconsistency when multiple events co-occur in the same article. Likewise, causal refinement is a structured pairwise decision problem that must remain conservative and verifiable, making a lightweight classifier with explicit consistency checks more suitable than free-form generation. Therefore, we intentionally use the LLM only for candidate trigger discovery, while relying on task-specific modules in later stages to preserve precision, controllability, and efficiency. Another relatively under-explored aspect is leveraging event relationships (such as causal and temporal links) during extraction. In many articles, events are not independent – one event may cause or enable another. For example, [Event₁: a wildfire broke out in California] triggers an Evacuation event. Identifying such relations could help fill information gaps; if two events share participants or locations due to a causal link, one event’s description can inform the other’s argument values. Prior works on event causality extraction focus on detecting cause-effect event pairs as a separate task, using classification or generation methods. However, this valuable signal has not been integrated into general event extractors. A notable exception is recent graph-based models that include event-event edges in their reasoning graph (e.g. event relationship graph in [Pan et al. \(2024\)](#)). Building on these insights, we incorporate causal reasoning into our extraction framework to improve coherence and recall.

In this paper, we propose a novel framework that addresses the above challenges by combining LLM reasoning, an agent-based iterative extraction strategy, and causal inference for robust document-level event extraction. Our contributions are summarized as follows:

1. We employ a large language model as a knowledgeable reader to improve trigger and event type identification. The LLM is prompted to scan the document and suggest candidate events, which are then verified by a smaller classifier. This hybrid approach achieves high recall in detecting diverse events, even when trigger clues are subtle or context-dependent. We intentionally confine the LLM to this front-end stage, because trigger discovery benefits most from broad semantic coverage, whereas downstream stages require stricter text grounding and schema control.
2. Rather than asking the LLM to directly generate a complete event record, we design a state-aware extraction agent that iteratively queries the document for the argument roles of each identified event. Instead of extracting all arguments in one pass, the agent maintains a partial event state, including the trigger span, event type, filled arguments, unfilled roles, and QA confidence scores. It initializes the role queue using schema-level priors, and then updates subsequent query formulation and context selection according to the spans and confidence scores returned by previous QA steps. This bounded feedback mechanism allows the agent to handle missing-role scenarios more gracefully while keeping argument prediction anchored to explicit source spans and reducing the risk that generative errors propagate across multiple roles and events.

3. Our framework explicitly detects potential causal relations between events in the same document using a lightweight classifier on trigger pairs. When a cause-effect link is found, we share or infer arguments between those events: for instance, an effect event might inherit the location or time of its cause if not mentioned otherwise. This causality-aware inference improves completeness and consistency of extracted event records under incomplete information conditions.
4. We conduct experiments on two datasets. Our model significantly outperforms eight strong baselines, including prior document-level frameworks and generative approaches by 4.3% average F1. It also exceeds a GPT-4 in-context learning baseline by a wide margin, confirming the advantage of our specialized architecture. We provide ablation studies demonstrating the contribution of each module (LLM, agent, causality) and additional analysis on extraction performance relative to document characteristics (event count, length).

The remainder of this paper is organized as follows. [Section 2](#) reviews related work on event extraction, including document-level extraction, use of LLMs, and agent/causal methods. [Section 3](#) formally defines the task and describes our proposed methodology and model components in detail. [Section 4](#) presents our experimental setup, datasets, baseline models, and reports quantitative results and discussions, including ablation and case studies. Finally, [Section 5](#) concludes with our contributions and future work directions, such as extending our approach to cross-document event extraction and real-time applications.

2. Related work

2.1. Event extraction foundations

Event extraction has long been studied in information extraction, with roots in the MUC conferences in the 1990s. The Message Understanding Conference (MUC) tasks included template filling for events such as terrorist attacks, laying the groundwork for defining events in text. MUC-5 (1993) ([Sundheim, 1991](#)) and subsequent evaluations introduced standard metrics and spurred research on rule-based and statistical methods. Later, the ACE 2005 program provided a multi-domain annotated corpus for event extraction, specifying event types and argument role schemas. Traditional approaches often adopted a pipeline: first perform event trigger detection (identifying the trigger word and classifying its event type), then perform argument role labeling (extracting text spans for each predefined role of that event type). Early systems used feature-based classifiers, kernel methods, or maximum entropy models for these subtasks, with limited ability to share information between them. [Ahn \(2006\)](#) described how complexity arises from trigger polysemy and implicit arguments, motivating joint inference.

The rise of machine learning and deep learning significantly advanced event extraction. On the trigger detection side, neural classifiers using CNNs and RNNs improved performance by capturing local context and lexical features automatically. For example, the CNN model of [Nguyen and Grishman \(2015\)](#) detected events and was extended via dynamic multi-pooling to better handle multiple triggers in a sentence ([Chen et al., 2015](#)). Recurrent models and sequence labeling approaches (e.g. bidirectional LSTMs) enabled joint encoding of sentences; [Nguyen et al. \(2016\)](#) trained a bi-directional RNN to jointly predict trigger and argument labels, achieving the first fully joint neural model on ACE. Subsequent innovations included graph-based models that encode dependency or semantic graphs of the sentence. [Nguyen and Grishman \(2018\)](#) integrate a graph convolutional network that propagates information between trigger and entity nodes, using argument-aware pooling to emphasize relevant words for event detection. [Liu et al. \(2018\)](#) and other works introduced attention mechanisms over graphs, effectively aggregating evidence for multiple events mentioned together.

Joint modeling of entities and events became a trend to exploit cross-dependencies. OneIE ([Nguyen & Nguyen, 2019](#)) is a notable joint model that represents the entire sentence's information (entities, triggers, roles,

relations) as a graph and searches for the globally optimal extraction result. By incorporating global features (e.g., a Victim of a Die event likely overlaps with a Victim of an Attack event in the same sentence), OneIE improved consistency across predictions. Similarly, [Wadden et al. \(2019\)](#) developed DyGIE++ which uses span representations and a graph message passing network to jointly extract entities, relations, and events from text, demonstrating benefits of multi-task learning for event arguments as well. These joint models blur the pipeline boundaries and leverage interactions between subtasks, which is beneficial for resolving ambiguities (e.g., distinguishing event types by the presence of certain arguments).

2.2. Document-level event extraction

Extending extraction to the document level has introduced new methods to handle cross-sentence context and multiple events. Early attempts like DCFEE [Yang et al. \(2018\)](#) addressed a specific domain (financial news) by aggregating sentence-level extractions, but more general solutions soon emerged. A key challenge is that arguments for an event might be mentioned in different sentences (or not at all). Another challenge is distinguishing multiple occurrences of the same event type in a document. For example, a news article may describe two separate Attack events; a model must group the correct arguments with the right trigger (often called argument-role clustering). One line of approaches formulates document-level extraction as a template filling or table filling problem. Doc2EDAG ([Zheng et al., 2019](#)) introduced the idea of constructing an event DAG: starting from each trigger, iteratively attach argument entities to build a complete event record. This approach can elegantly handle multiple events of the same type by forming multiple DAG instances. A related idea is to use a predetermined table of slots for each event type and fill it with text spans from anywhere in the document. For instance, the TabEAE framework ([He et al., 2023](#)) treats the task as filling a table of size (roles $\times$ events) for each document, enabling parallel extraction of all event mentions. These template-based methods inherently accommodate heterogeneous argument structures, since each event type has its own schema of slots.

Another line of work employs graph neural networks (GNNs) and heterogeneous graphs to model long documents. [Xu et al. \(2021\)](#) build a graph with different node types (sentences, entities) and connect nodes based on entity mentions in sentences, then perform iterative message passing to propagate information across sentence boundaries. This proved effective for linking arguments spread across sentences. [Liang et al. \(2022\)](#) extend this by adding explicit event nodes and event-event edges (for events in the same document), capturing event relationship signals. [Wang et al. \(2023a\)](#) similarly use event “proxy nodes” to gather mentions of the same event and apply a spatial learning objective (Hausdorff distance) to better separate distinct events. These graph-based approaches excel at modeling scattered arguments and coreference – for example, if “the president” in one sentence and a person’s name in another refer to the same individual, the graph can unify them for the event’s Agent role. There are also decoding strategy innovations. [Yang et al. \(2021\)](#) proposed a non-autoregressive parallel decoder that generates all events in the document simultaneously, using a matching loss to assign predicted arguments to the correct event slot. This avoids the sequential bias of autoregressive decoding and speeds up extraction. Some works have treated document-level extraction as a set partitioning or clustering problem – e.g., first identify all candidate arguments and then cluster them by event triggers (or vice versa). For instance, [Zhu et al. \(2022\)](#) devised a proxy-based complete graph (PTPCG) where they create cliques of arguments that belong to the same event, using a pruning strategy to reduce combinations.

In summary, document-level event extraction approaches strive to integrate information across the document. They often require more complex inference than sentence-level models, and the design of global features or constraints (e.g. each event must have at most one Location argument) is critical. Our work builds on these advancements by intro-

ducing causal links as a new source of global signal and by using an agent strategy to retrieve arguments across the document.

2.3. Large language models for information extraction

Large pre-trained language models have revolutionized NLP tasks, and researchers have begun investigating their application to event extraction. One straightforward approach is using LLMs in an in-context learning paradigm: given a prompt that explains the task and a few examples, the LLM generates extraction outputs for a new input. For example, recent studies have prompted GPT-3 or GPT-4 with formatted demonstrations to extract events from text. However, results show that without fine-tuning, LLMs often miss details or produce overly verbose outputs not suitable for structured extraction. [Chen et al. \(2024\)](#) report that even strong LLMs underperform smaller task-specific models on benchmark event extraction datasets, especially in multi-event documents.

To better leverage LLM capabilities, another direction is instruction tuning and fine-tuning on IE tasks. For instance, [Wang et al. \(2023b\)](#) (InstructUIE) fine-tuned a T5-based model on a mixture of IE tasks (entity, relation, event) using natural language instructions for each task, achieving a unified IE model that can handle event extraction among others. Similarly, researchers have fine-tuned GPT-style models on event data with task descriptions as additional context. [Ma et al. \(2022\)](#) introduced prompt-based fine-tuning for event argument extraction: by crafting prompts that include role names (e.g. “The Place of the event is [MASK]”), they convert extraction into a masked language modeling problem, improving performance on RAMS and WikiEvents. [Hsu et al. \(2021\)](#) and its extension ([Hsu et al., 2023](#)) take a generation-based view, where the input document plus some prompt text is fed into a language model (BART or T5) which is trained to output a structured list of arguments. They even integrate abstract meaning representation (AMR) knowledge into the prompt (AMPERE model) to guide the generation of arguments coherently.

An alternative use of LLMs is as a knowledge source or annotator. For example, [Lin and Chen \(2021\)](#) explored eliciting knowledge from GPT-3 by asking it questions relevant to event arguments (like “who might be the perpetrator?”) and using the answers to assist a smaller model. [Yu et al. \(2024\)](#) generate a chain-of-thought (rationale) from GPT-3.5 for each training sentence explaining why a particular argument is correct, and use these rationales to augment training data for a smaller event extractor. Their framework (EE-LCE) showed that the additional reasoning signal from LLM can improve the model’s generalization. Our approach in this paper similarly uses an LLM to enhance reasoning: we use it to identify potential triggers and to possibly bridge long-distance context for arguments. However, we keep a deterministic extractive backbone to ensure that output remains grounded in the input text and follows the required structured format.

2.4. Agent-based and causal reasoning approaches

There is relatively limited prior work on agent-based extraction in NLP, but a few notable studies exist. An agent-based approach treats information extraction as a sequential decision process, where an agent “reads” the text and decides what to extract step by step. For instance, [Zhao et al. \(2021\)](#) proposed a reinforcement learning agent that extracts multiple biomedical events by learning an optimal order to extract triggers and arguments, yielding a more efficient extraction than exhaustive search. Another creative work by [Du and Cardie \(2020\)](#) designed a dialogue-based framework: one agent asks questions about potential arguments and another agent (or the text context) answers, effectively performing extraction through Q&A dialogue. They applied this to a conversation setting, but the concept can translate to an agent iteratively querying a document. These approaches show that agent strategies can be beneficial when the extraction task is complex, as the agent can break it down and focus on one piece of information at a time. Inspired by this,

our framework includes an Argument Extraction Agent that goes role by role, which to our knowledge is a novel design for document-level event extraction. By formulating each argument search as a query (which can be answered by either a smaller model or even by the LLM), the agent mimics how a human might systematically scan a document for specific details.

On the other hand, causal reasoning in event extraction has mostly been studied as identifying event-event relationships. There are dedicated datasets and competitions for event causality extraction, where the goal is to detect if one event mention is a cause of another. Traditional methods classify pairs of event mentions with features or neural models, while newer approaches use generation with templates to output cause-effect pairs. Importantly, these works assume gold event mentions as input; they do not integrate the causality detection with the event extraction itself. One exception is [Glavaš et al. \(2014\)](#) who jointly extracted events and causal links from news, but using classical pipelines. Very recent work by [Liu et al. \(2023\)](#) introduced a framework (ICE) that infers implicit cause-effect interactions by using ground-truth event information during training, but it still treats the extraction of events and extraction of causal links as separate outputs. Our approach is the first to use causal relations to aid the extraction of the events themselves. By identifying causally related events within a document, we impose a consistency: e.g. a cause and its effect might share a common location or participant (especially in news stories where one incident leads to another). We encode this intuition by propagating arguments between linked events. This is somewhat analogous to using co-reference links to ensure two mentions share an entity argument, but here the link is a narrative causal link.

In summary, while prior work has made great strides in document-level event extraction, our work distinguishes itself by combining three aspects: utilizing LLM capabilities in a controlled way, structuring the extraction process as an interactive agent, and incorporating event-event causal knowledge. In the next section, we formally define our task and then present our methodology built upon these ideas.

3. Methodology

In this section, we first formalize the task of multi-type heterogeneous argument event extraction. We then provide an overview of our proposed architecture, followed by detailed descriptions of each component: the LLM-based Event Detector, the Agent-based Argument Extractor, and the Causal Inference Module. Finally, we describe the training objectives and algorithm.

3.1. Task definition

We define the input as a document (a news article or report) consisting of multiple sentences:

$$D = \{s_1, s_2, \dots, s_n\}. \quad (1)$$

The output is a set of events

$$\mathcal{E} = \{E_1, E_2, \dots, E_m\} \quad (2)$$

mentioned in the document. Each event E_i is represented as a tuple:

$$E_i = (\text{Type}_i, \text{Trigger}_i, \{(r_{i1} : A_{i1}), (r_{i2} : A_{i2}), \dots\}), \quad (3)$$

where Type_i is the event type (from a predefined ontology of event types), Trigger_i is the text span (one or more words) in the document that most clearly expresses the occurrence of the event, and $\{(r : A)\}$ is a set of argument role–argument value pairs for that event. Each role r is a slot defined by the event schema (e.g., Agent, Victim, Time, Location, Cause, etc.), and A is either a text span from the document filling that role or a special null value if the argument is not explicitly present. Heterogeneous arguments means that the set of roles (and their quantity) varies depending on the event’s type. For example, a Natural Disaster event might have roles {Location, Date, Casualties, Cause}, whereas a

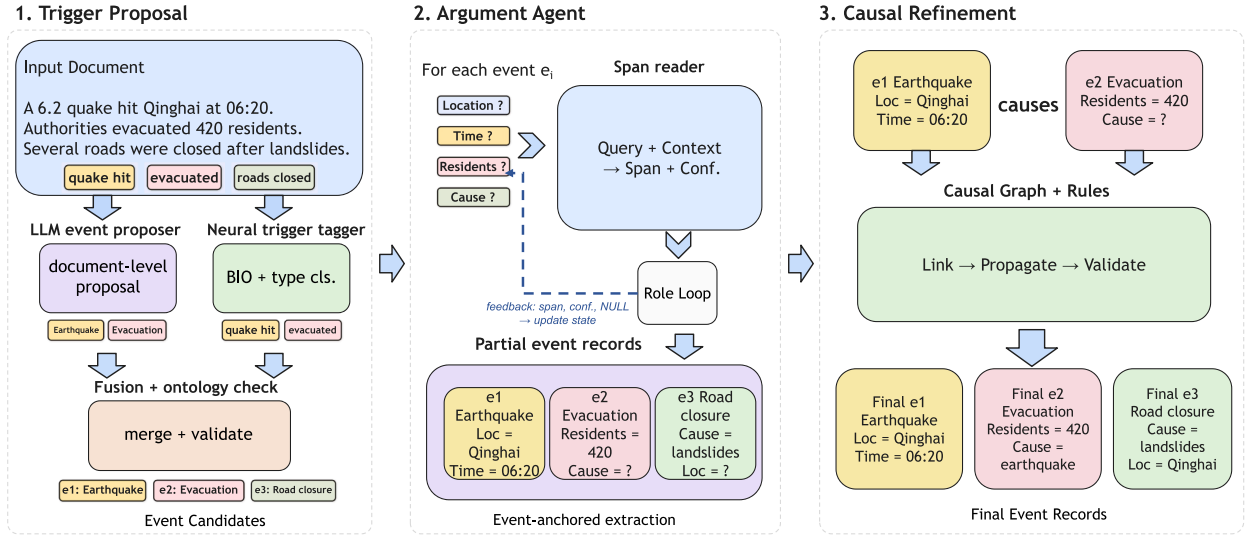


Fig. 1. Overview of the proposed causality-aware LLM-Agent framework with a running example from a news document. The left part shows two complementary trigger extraction sources: an LLM event proposer that scans the full article and a neural trigger tagger that performs local BIO-based trigger and type prediction. Their outputs are merged by a fusion and ontology-check step to form candidate events. The middle part illustrates the event-wise argument extraction agent, where each candidate is treated as a trigger-anchored event instance rather than only an event type. The agent issues role-specific queries to a span reader over the local context tied to the current trigger and the LLM-highlighted clues associated with that candidate, producing grounded partial event records. The right part shows causal refinement, where a causal classifier identifies event-event links and refinement rules propagate compatible arguments, fill missing Cause information, and validate conflicts to obtain the final document-level event records.

Personnel Appointment event might have {Person, Position, Organization, Date}. We denote the role set for event type T as $\mathcal{R}(T)$.

The task requires identifying all event instances in the document and extracting as many of their defined arguments as possible from the text. An argument is considered correctly extracted if the text span A exactly matches the reference span in the gold annotation and it is assigned the correct role r for the correct event. We also consider a trigger correctly identified if its span and event type match the gold trigger. The evaluation typically computes Precision and Recall on the collection of extracted argument-role instances (and triggers) across all documents. A high-quality solution should have both high precision (avoid spurious or incorrectly classified events/arguments) and high recall (capture events even if information is partially missing or spread out).

3.2. Overall architecture

Fig. 1 presents a concrete running example of the proposed framework and makes the internal logic of the pipeline explicit. Starting from a news document, the left part highlights how complementary trigger proposals from the LLM event proposer and the neural trigger tagger are fused through an ontology-aware merging step to form candidate events. The middle part shows how each candidate event is processed by the argument extraction agent through state-aware role-wise span queries. The agent updates its partial event state after each QA call and uses the returned span and confidence score as feedback for subsequent query formulation and context selection, yielding grounded partial event records. The right part further illustrates how causal refinement links related events, propagates compatible information, and produces the final document-level event records. The system consists of three main modules that operate in sequence:

- **Event Detection Module (LLM-Augmented):** This module takes the document D and identifies candidate events. It has two sub-components: a prompt-based query to an LLM and a fine-tuned classification model. First, we prompt the LLM with an instruction to list events in the document, for example: “Read the following news article and identify the events mentioned (with their type and trigger words).” The LLM’s output is parsed to extract trigger word candidates and tentative types. This serves as a high-recall but possibly

low-precision suggestion. Meanwhile, our trained Event Classifier (a BERT-based model) scans each sentence to detect triggers in a more controlled manner (using BIO tagging for trigger spans and a classification head for type). The outputs of these two are fused – specifically, any trigger detected by either is taken as a candidate, but the final event type is assigned by the classifier (unless the LLM suggests a type for a trigger that the classifier missed, in which case we use the LLM’s suggestion). The result is a set of predicted events $\{\hat{E}_1, \hat{E}_2, \dots, \hat{E}_k\}$ with trigger spans and types. We deliberately place the LLM only in this stage because candidate trigger discovery is recall-sensitive and benefits most from document-level semantic abstraction, while later stages require tighter grounding to specific spans and stricter structural control.

- **Argument Extraction Agent:** For each predicted event $\hat{E}_i = (\hat{y}_i, \hat{t}_i, \cdot)$, an Argument Extraction Agent module is invoked to fill its arguments. Instead of using a purely fixed role sequence, the agent maintains a lightweight extraction state for the current event, including its event type, trigger span, current context window, filled arguments, unfilled roles, and QA confidence scores. The role queue is initialized according to schema-level priors, and each role is queried by a natural language question, for example: “What is the <Location> of the <Disaster> event triggered by ‘explosion’?” We then use a span-based reader model, implemented as a RoBERTa-based QA model, to return an answer span and a confidence score. After each QA call, the agent updates the event state. If the answer is confident, the span is stored as the corresponding argument; if the answer is NULL or low-confidence, the role is marked as unresolved and the agent can perform a bounded fallback step by expanding the local context with LLM-highlighted clues and reformulating the query using already filled arguments. We intentionally use a span-based reader rather than an LLM generator here, because argument extraction requires boundary-exact, role-conditioned evidence anchored to the source document. This design reduces hallucinated fillers and keeps the output consistent with the event schema when multiple events coexist in the same article. The outcome of this module is a partially or fully filled event record

$$\hat{E}_i = (\hat{y}_i, \hat{t}_i, \{(r : \hat{a}_{i,r})\}). \quad (4)$$

• **Causal Inference Module:** After extracting individual events, this module analyzes relations between them to refine the results. We likewise avoid free-form LLM generation in this stage, because causal refinement is a structured event-pair decision problem where conservative classification and explicit consistency checks are easier to calibrate, interpret, and verify. It uses a simple pairwise classifier that takes representations of two events (we use the average BERT embeddings of their trigger words and argument head words) and predicts if one event is the cause of the other. If a causal relation is detected (say $\hat{E}_p$ is predicted to cause $\hat{E}_q$), we then compare their argument sets. We merge complementary information: for any role r that is unfilled in E_q but filled in E_p (and semantically, it makes sense that they would share this role), we assign $\hat{E}_{q,r} := \hat{E}_p.r$ as an inferred argument. For example, if a Cause event (like an earthquake) has a location argument “California” and the Effect event (power outage) lacks a location, we propagate “California” as the location of the outage. We do this propagation for generic roles like Location, Time, and common participant roles. Additionally, the causal module can validate existing arguments: if two linked events have conflicting entries for what should be the same entity (e.g., two different locations), it flags low confidence. The inferred or adjusted arguments are added to produce the final event set output.

The three modules are connected in a pipeline but also share information: Module A’s LLM highlighting helps Module B’s agent focus, and Module C’s inferences can loop back to update arguments (we allow one iteration of refinement for each causal link found).

Formally, our model’s extraction can be viewed as optimizing a joint objective that factors into these modules. For an event $E = (y, t, \cdot)$, each event is scored by:

$$s(E) = s_{\text{trigger}}(t, y) + \sum_{r \in \mathcal{R}(y)} s_{\text{arg}}(a_r \mid r, t, y) + s_{\text{cause}}(E, \text{Causer}(E)). \quad (5)$$

where s_{trigger} is the classifier score for the trigger and type, $s_{\text{arg}}(A_r)$ is the QA model logit for choosing span A_r as role r ’s filler (or a default score if none), and s_{cause} is a bonus for coherent causal linking (we assign a small positive score if an event has a causal link with arguments aligned, encouraging those configurations). Our system can be seen as finding the set of events $\hat{E}$ that maximizes the total score across all events. In practice, we greedily select high-scoring triggers, fill arguments greedily, and then adjust with causal rules. This approximate decoding is effective and much more tractable than a brute-force joint search over all combinations of arguments.

3.3. LLM-based event detection

We intentionally place the LLM only in the event detection stage rather than throughout the full pipeline. Trigger identification in news documents is primarily a recall-oriented semantic discovery problem: event mentions may be paraphrased, implicit, or supported by long-range discourse context, so the LLM is most useful as a knowledgeable reader that proposes plausible candidates from the full document. Once an event candidate has been identified, however, the downstream tasks become more constrained: argument extraction requires exact source spans for predefined roles, and causal refinement requires conservative pairwise decisions with explicit consistency checks. For these later stages, smaller task-specific models are more reliable, easier to calibrate, and less prone to hallucination than free-form LLM generation.

Neural Trigger Classifier: We fine-tune a BERT-based model for trigger identification. Specifically, we use a multi-task setup: a token-level BIO tagging for trigger spans and a classification of the trigger’s event type. During training, any token that begins a trigger is tagged B- $\langle \text{Type} \rangle$, and subsequent tokens of that trigger as I- $\langle \text{Type} \rangle$. We also mark non-trigger tokens as O. We optimize a cross-entropy loss on these tags. At inference, the model labels each token. We collect predicted trigger spans (continuous B-I sequences) and take their label as the event type. This model alone provides high-precision trigger detection, but

might have lower recall if triggers are metaphorical or if the trigger word is uncommon (lack training examples). That’s where the LLM helps by possibly suggesting those.

Fusion: We denote the classifier outputs as $T_c = \{(t_i^c, y_i^c, s_i^c)\}_{i=1}^{n_c}$ and the LLM proposals as $T_l = \{(t_j^l, y_j^l, s_j^l)\}_{j=1}^{n_l}$, where s denotes the sentence index of the trigger. For each $(t_i^c, y_i^c, s_i^c) \in T_c$, we create a trigger-anchored event candidate $\hat{E}_i = (id_i, \hat{y}_i, \hat{t}_i, \hat{s}_i, \cdot)$. Thus, even if two triggers have the same event type, they are preserved as different event instances as long as their trigger spans or sentence positions differ. For each $(t_j^l, y_j^l, s_j^l) \in T_l$ that is not matched by the classifier, we retain the candidate only if y_j^l belongs to the predefined ontology and the proposal contains an explicit trigger span. Two candidates are merged only when their event types are consistent and their trigger spans overlap or refer to the same textual mention; otherwise, they remain separate event instances.

This trigger-anchored fusion strategy also limits the effect of possible LLM merging or splitting errors. If the LLM merges two closely related events into one broad proposal, the classifier-detected trigger anchors are still kept as separate candidates, and the LLM proposal is used only as auxiliary contextual evidence rather than as a reason to collapse the candidates. Conversely, if the LLM splits one event into several proposals but their trigger spans overlap and their types are consistent, they are deduplicated into one trigger-anchored candidate. In this way, the LLM contributes document-level semantic recall at the front end, while the downstream argument extraction is performed over grounded event instances rather than type-level event pools.

3.4. Agent for argument extraction

Once we have a predicted event $\hat{E} = (\hat{y}, \hat{t}, \cdot)$, we need to extract its arguments. A naive approach would run a role labeler over every token in the document (classifying if that token starts an argument of a given role). However, this is inefficient and prone to false positives given the long context. Instead, we deploy the Argument Extraction Agent, which focuses on one role at a time and uses the context strategically.

The agent maintains a lightweight extraction state for the current predicted event instance. The state includes the event id, event type $\hat{y}$, trigger span $\hat{t}$, trigger sentence index $\hat{s}$, the current trigger-anchored context window, the set of filled arguments, the set of unfilled roles, and the confidence scores returned by the QA reader. Therefore, two events with the same type are not processed as a shared type-level record; they are processed as separate trigger-anchored instances with independent argument dictionaries and independent extraction states. We initialize a role queue with all roles in $\mathcal{R}(\hat{y})$ according to schema-level priors: roles such as Time and Location are assigned high initial priority because they are often explicitly stated, while roles such as Cause or Purpose are queried after more event-specific evidence has been collected. Importantly, this initialization does not mean that the extraction process is a purely fixed sequence. After each QA call, the agent updates the state of the current event instance and uses the returned span, confidence score, and NULL decision as feedback to adjust subsequent query formulation and local context selection.

For each selected role r , the agent constructs a query $q(r, \hat{E}, S)$ conditioned on the current event state S . The query includes the event type $\hat{y}$, trigger span $\hat{t}$, and, when available, already filled arguments. For instance, if $\hat{E}$ is of type Attack with trigger “opened fire”, and $r = \text{Perpetrator}$, the initial query can be: “In the event where someone opened fire (Attack event), who was the perpetrator?” If another role such as Victim or Location has already been filled, the query can be reformulated to include this evidence, thereby focusing the QA reader on the relevant event instance. This natural question can be understood by a QA model that has been fine-tuned on extraction tasks. We fine-tuned a RoBERTa-large model on a combination of ACE and the competition data formulated as QA: each argument annotation was turned into a question-answer pair. This model learns to find the text span answering the question, or output NULL if no answer is supported by the context.

We restrict the context given to the QA model to a window of text likely containing the answer. By default, we take the sentence containing the current trigger and the adjacent sentence before and after it. If the LLM in module A underlined or mentioned certain sentences as related to this event, we include only the highlighted clues associated with the current trigger-anchored candidate. When two triggers of the same type appear in nearby sentences, their context windows may overlap, but their argument states are not shared. Each QA query is conditioned on the current event anchor, including the event type, trigger span, and trigger sentence. The returned span is accepted only for the current event instance if it is compatible with this anchor; otherwise, it is rejected or treated as low-confidence.

The QA model then returns an answer span A_r and a score. If the score is above threshold τ (we set $\tau = 0.5$ on a normalized 0–1 scale), we accept A_r as the argument for role r and update the current event state. If the score is below τ or the answer is NULL, the agent does not simply discard the role and move on. Instead, it marks the role as unresolved and performs a bounded fallback step: it expands the local context with LLM-highlighted clues associated with the same event anchor and reformulates the query using the current partial event record. For example, if the Time role is missing, the agent re-queries the document using the trigger sentence together with available Location, participant, or causal clues, rather than only following the original role list. If the fallback query still returns a low-confidence or NULL answer, the role is left empty and may later be handled by the causal refinement module. Algorithm 1 outlines this state-aware and trigger-anchored extraction process.

Algorithm 1 State-aware and trigger-anchored argument extraction agent.

Input: Document D , predicted event instance $\hat{E} = (id, \hat{y}, \hat{t}, \hat{s}, \cdot)$, role set $\mathcal{R}(\hat{y})$, confidence threshold τ

Output: Filled arguments ArgDict for $\hat{E}$

```

1: ArgDict  $\leftarrow \{\}$ 
2: Anchor  $\leftarrow (\hat{y}, \hat{t}, \hat{s})$   $\triangleright$  event type, trigger span, and trigger sentence
3: State  $\leftarrow \{\text{Anchor, ArgDict, unfilled roles, confidence scores}\}$ 
4: RoleQueue  $\leftarrow \text{InitializeQueue}(\mathcal{R}(\hat{y}))$ 
5: ContextIndex  $\leftarrow \text{GetAnchoredContextWindowIndices}(D, \text{Anchor})$ 
6: Highlighted  $\leftarrow \text{FilterLLMHighlight}(\hat{E}, \text{Anchor})$ 
7: ContextSentences  $\leftarrow \text{BuildAnchoredContext}(\text{ContextIndex}, \text{Highlighted})$ 
8: while RoleQueue is not empty do
9:    $r \leftarrow \text{SelectNextRole}(\text{RoleQueue}, \text{State})$ 
10:  question  $\leftarrow \text{FormulateQuery}(r, \text{Anchor}, \text{ArgDict})$ 
11:  (answer_span, score)  $\leftarrow \text{QA\_Model.answer}(\text{question}, \text{context} = \text{ContextSentences})$ 
12:  score  $\leftarrow \text{VerifyEventAnchor}(\text{answer\_span}, \text{score}, \text{Anchor})$ 
13:  if score  $< \tau$  or answer_span = NULL then
14:    ContextSentences'  $\leftarrow \text{ExpandAnchoredContext}(D, \text{Anchor}, \text{Highlighted}, \text{ArgDict})$ 
15:    question'  $\leftarrow \text{ReformulateQuery}(r, \text{Anchor}, \text{ArgDict})$ 
16:    (answer_span, score)  $\leftarrow \text{QA\_Model.answer}(\text{question}', \text{context} = \text{ContextSentences'})$ 
17:    score  $\leftarrow \text{VerifyEventAnchor}(\text{answer\_span}, \text{score}, \text{Anchor})$ 
18:  if score  $\geq \tau$  and answer_span  $\neq$  NULL then
19:    ArgDict[ $r$ ]  $\leftarrow$  answer_span
20:  else
21:    ArgDict[ $r$ ]  $\leftarrow$  None
22:  State  $\leftarrow \text{UpdateState}(\text{State}, r, \text{ArgDict}[r], \text{score})$ 
23:  RoleQueue  $\leftarrow \text{UpdateQueue}(\text{RoleQueue}, \text{State})$ 
24: return ArgDict

```

This agent allows bounded adaptive querying within each event instance. The initial role queue is heuristic, but the execution is feedback-guided rather than a purely fixed QA script. After each QA call, the

agent observes the returned span, confidence score, and NULL decision, updates the partial event state, and then uses this state to adjust subsequent queries and context selection. For example, if the Cause role is queried after several participant or location roles have been filled, the query can be conditioned on these already extracted arguments. Similarly, if the Time role is missing or low-confidence, the agent can expand the context around the trigger and reformulate the query using available Location, participant, or causal clues before deciding to leave the role empty. One important detail is that if an argument role is typically filled by one of a set of entity types, such as person or organization, we incorporate that expectation into the query. For example, for Victim, we may add “name or identity of the victim” to guide the QA model. This was found to improve precision slightly by biasing the model toward choosing person names for victim roles.

We further apply an event-instance anchoring strategy to reduce argument cross-pollination. This is not intended to be a full neural event coreference resolver; rather, it is a conservative control mechanism for window-based QA. Each predicted event is represented by its event type, trigger span, and trigger sentence index. Candidate arguments extracted from overlapping windows are assigned only to the current event instance if they are compatible with this anchor. If two same-type events occur in the same document, their argument dictionaries remain separated throughout extraction. If the LLM proposer produces a merged event description, the classifier-derived trigger anchors are preserved and the LLM output is used only as auxiliary evidence. This design reduces the risk that arguments from one event instance are incorrectly attached to another event of the same type.

3.5. Incorporating causal inference

After populating arguments for events independently, we introduce the Causal Inference Module to leverage inter-event relationships. We hypothesize that if event E_p is the cause of event E_q , then certain arguments of E_p can help complete E_q . To find cause-effect pairs, we use a simple classifier $f_{\text{cause}}(E_i, E_j)$ that takes two events and outputs a probability $P(\text{Cause} = 1 | E_i, E_j)$. We consider basic features: the event types (certain types like Disaster often cause others like Damage), the textual distance between triggers (a cause often appears before effect), and explicit causal indicators (if the text between or linking the two sentences contains words like “because” or “led to”). Rather than engineering features, we fine-tuned a RoBERTa model on an auxiliary dataset of causal event pairs (we used a portion of the CCKS-2020 causality data for training). This model achieved reasonable accuracy (85% F1) on held-out causal pairs. We denote the causal-link decision threshold by τ_{cause} and fix $\tau_{\text{cause}} = 0.5$ in all experiments. We apply this model to each unordered pair of extracted events from the document. If both $f_{\text{cause}}(E_i, E_j) > \tau_{\text{cause}}$ and $f_{\text{cause}}(E_j, E_i) < \tau_{\text{cause}}$ (indicating E_i causes E_j), we mark $E_i \rightarrow E_j$ as a causal link. We ignore cases where two events might mutually cause each other (rare) or the classifier is uncertain.

Once we have a set of causal links, we perform argument inference. Specifically, for each link $\hat{E}_p = (\hat{y}_p, \hat{t}_p, \cdot) \rightarrow \hat{E}_q = (\hat{y}_q, \hat{t}_q, \cdot)$:

- For each role $r \in \mathcal{R}(\hat{y}_q)$ that is empty in $\hat{E}_q$, we check whether $r \in \mathcal{R}(\hat{y}_p)$ and whether $\hat{E}_p$ has a non-empty value for the same role. If so, we copy that value from $\hat{E}_p$ to $\hat{E}_q$. Rationale: the cause event’s argument can serve as a default for the effect if the effect does not specify it. This is especially true for roles like Time (if an explosion caused a fire, and the fire’s time is not stated, it likely started soon after the explosion – we might approximate the time as the same, though here we only copy exact spans, which might or might not be precise).
- Additionally, if the role Cause belongs to $\mathcal{R}(\hat{y}_q)$ and is empty in $\hat{E}_q$, we fill it with the trigger span $\hat{t}_p$ or a short description derived from the cause event $\hat{E}_p$. This effectively links the two in output.
- Conversely, if the schema of $\hat{E}_p$ contains an Effect or Consequence role, we may fill it using the trigger span $\hat{t}_q$.

We ensure not to overwrite any arguments that the agent already found; we only fill ones that were missing (None). More concretely, the consistency check applies three constraints before a propagated filler is accepted. First, the target role must belong to the schema of the effect event and must still be empty; we never overwrite an argument that was directly extracted by the agent. Second, propagation is restricted to role types for which sharing is semantically plausible, such as Location, Time, Cause, and shared participant roles, rather than being applied indiscriminately to all roles. Third, we compare the propagated filler against the target event’s local context window (including the trigger sentence and the nearby sentences used by the QA module). If this local context already contains an explicit but different filler for the same role, the propagated argument is rejected or marked uncertain. In this way, the consistency check is designed to block false positive propagation even when the causal classifier proposes an incorrect or overly broad relation.

This causal integration is a simple form of multi-event joint inference. It does not require retraining the main model with causal data – it’s a rule-based post-processing guided by a classifier. We found that it modestly improves recall (some previously empty roles get filled correctly) and consistency. For example, in one validation article about a storm (cause) leading to a power outage (effect), our agent initially missed the location of the outage because it was mentioned only in the context of the storm. After causal linking, the location “Florida” from the storm was assigned to the outage event’s location role, which matched the gold annotation.

3.6. Model training and objectives

Each component of our framework is trained separately on appropriate data:

- The Event Trigger Classifier is trained on annotated triggers (from training documents) to minimize cross-entropy on trigger start/end and type classification. We use the Adam optimizer with learning rate $2e-5$, and apply dropout of 0.3. The classifier achieves about 0.85 F1 on trigger identification on the dev set, providing a strong starting point.
- The Argument QA Extractor is fine-tuned on a combination of ACE 2005 and the competition’s training set converted to QA format. We generate one QA pair for each role of each event in the training data. If an argument is not present, we include a QA pair where the expected answer is null. We found including negative examples (no answer) is important so the model learns to abstain when appropriate. The QA model (RoBERTa-large) is fine-tuned for 3 epochs with learning rate $1e-5$. Its answer span prediction accuracy for known arguments was 0.80 F1 on dev (span match).
- The Causal Classifier is fine-tuned on an external corpus of event relations: we used 5000 cause-effect event pairs (and an equal number of non-causal event pairs) from a Chinese event causality dataset. For News15, only the short sentence pairs used by this auxiliary causal classifier are translated into English so that they can be encoded by the English RoBERTa classifier. This local translation step is confined to the causal module and is not used in trigger detection, argument extraction, or evaluation. For WikiEvents, the causal classifier operates directly on the original English sentence pairs. It’s a binary classifier on the concatenation of two event trigger sentences with a separator. This achieved 90% accuracy on held-out pairs and we observed it generalizes to our data (which is similar news domain).

We integrate these modules without end-to-end training. An end-to-end joint training would be challenging due to the complexity of the search process (the agent’s iterative decisions are not differentiable). Instead, we rely on pipeline training. During development, we tuned the fusion threshold for LLM suggestions and QA answer confidence on a validation set to maximize overall event extraction F1. Our final objective is the macro-averaged F1 over all role-instance extractions (includ-

ing trigger as a special instance). We do not explicitly train on this measure, but our design choices (like thresholding and causal propagation rules) were guided by maximizing F1 on dev. The next section details the experimental results, demonstrating how each module contributes to improved performance.

4. Experiments

We evaluate our proposed approach on two datasets to verify its effectiveness across languages and domains. We compare against a broad range of baseline methods and conduct ablation studies to understand the impact of each component. All experiments are conducted on a workstation with NVIDIA A100 GPUs.

4.1. Datasets

News15 Dataset: This is a document-level event extraction dataset derived from a recent competition on multi-type event extraction in Chinese news. It contains 15 event types covering domains like public security incidents, natural disasters, financial events, and so on (e.g., TrafficAccident, Earthquake, CorporateAcquisition, etc.). Each event type comes with a predefined set of argument roles. The dataset provides the full text of each news article and annotations of all events mentioned. On average, each article has 1.7 events, with some containing up to 4 events. The documents are relatively long (average 300 Chinese characters) and often semi-structured (some with bullet points or lists). Throughout the main extraction pipeline, News15 is kept in its original Chinese form; no document-level translation is applied during trigger detection, argument extraction, or evaluation. Any English rendering in the paper is only for exposition. Importantly, not every event mention contains all possible roles – for instance, Investigation events often lack an explicit Outcome argument in the text. We used the official split: 6,000 documents for training, 2000 for development, and 4000 for testing. We note that this dataset’s annotations expect exact span matches for arguments; if an argument is mentioned multiple times, any matching span is considered correct.

WikiEvents (English): WikiEvents is a public dataset introduced by Li et al. (2021) for document-level event extraction. It consists of 246 English documents (206 for training, 20 for development, and 20 for testing) annotated for 50 event types adapted from the ACE and Rich ERE ontologies. WikiEvents is challenging because the documents are longer (average 800 tokens) and some event types are quite fine-grained or infrequent. Most events in WikiEvents have their arguments contained in one or two contiguous sentences (94% of arguments are in the same sentence as the trigger), but cross-sentence context is still needed for the remaining cases. We use WikiEvents to test our model’s generalization to English and to compare with published results.

We did not include the RAMS dataset explicitly in main experiments because it focuses on single event per document scenario with a large ontology (139 types), but we do use it indirectly for training the QA model (some RAMS instances were included in our QA fine-tuning data).

4.2. Evaluation metrics

For both datasets, we evaluate using Precision (P), Recall (R), and F1-score (F1) for two levels:

- **Trigger identification & classification:** A trigger prediction is correct if its span and event type match a reference trigger.
- **Argument role extraction:** An argument is correct if the span and role label match one of the gold arguments for some gold event of the same type. (If multiple events of the same type exist, each argument is tied to the specific event instance – we require correct association to the right event.)

Following common practice, we primarily report the argument-level F1 as the main metric for overall extraction quality. We also ensure that

an extracted argument is only considered if the corresponding trigger and type were correctly predicted (making it a strict measure of full event extraction). We adopt micro-averaged precision, recall, and F1 across all documents. A true positive for an argument role is counted when there is a predicted span-role that matches a gold span-role for the same event instance; otherwise, it's a false positive (if predicted with no match) or false negative (if gold not predicted). We calculate P, R, F1 for each dataset and also break down by event type in some analysis. For significance testing, we use a paired bootstrap test at 95% confidence on F1.

4.3. Experimental settings

Our code is implemented in Python using PyTorch. We use HuggingFace Transformers for BERT, RoBERTa, and GPT integration. The BERT-based trigger classifier is a BERT-base Chinese model for News15 and BERT-base cased for WikiEvents (English). The QA model is RoBERTa-large (multi-lingual XLM-R for Chinese data), fine-tuned as described earlier. For GPT-4 prompting, we use OpenAI's API (gpt-4-0314 model) with a max token limit of 512 for output; to ensure determinism in experiments, we use temperature 0 (greedy decoding). The causal classifier is RoBERTa-base. Unless otherwise noted, we fix the QA confidence threshold to $\tau_{QA} = 0.5$ and the causal-link detection threshold to $\tau_{cause} = 0.5$ throughout all experiments.

During inference, efficiency was an explicit design consideration. A naive sequential implementation of role-wise argument extraction would become slow as the number of event roles increases. To mitigate this, we execute independent QA queries for different roles in parallel threads and aggregate the results afterward, which substantially reduces wall-clock latency in practice. We also note that the causal module performs only one bounded post-extraction refinement pass over detected causal links, rather than unbounded multi-round iterative inference. On average, our system processes a document in about 1.2 s (including GPT calls, which are the slowest part). Without GPT (i.e., using only the neural trigger classifier), it takes about 0.8 s per document on a single GPU. A more detailed latency and computational complexity analysis is provided later in this section.

4.4. Baseline models

We compare our model with eight baseline methods representative of traditional pipelines, sentence-level joint extraction, document-level structural models, and end-to-end extraction approaches. In particular, to position our cascaded framework against end-to-end alternatives, we explicitly include Doc2EDAG and DE-PPN as representative document-level end-to-end baselines, and Text2Event as a representative end-to-end generative baseline.

- **DCFEE (Yang et al., 2018)** – A pipeline specifically for document-level financial events. We adapt it to our data by first extracting sentence-level events (using a BiLSTM-CRF for triggers and arguments) then performing a heuristic key event aggregation. This represents a classic pipeline without global inference.
- **Doc2EDAG (Zheng et al., 2019)** – An end-to-end document event extractor that generates event argument graphs. We use the authors' open-source implementation and train it on each dataset's training set. It inherently handles argument scattering via its graph.
- **OneIE (Lin et al., 2020)** – A joint extraction model for sentence-level IE. We use it as a strong sentence-level baseline by applying it to each sentence of the document and merging the outputs. (It doesn't do cross-sentence, so its recall is expected to suffer, but precision is high.)
- **DE-PPN (Yang et al., 2021)** – An end-to-end document-level model with parallel decoding. We retrain it on our data. Because it outputs multiple events in a single forward pass, it serves as a strong end-to-end baseline for comparison with our cascaded framework.

- **GIT (Xu et al., 2021)** – The graph interaction model with tracker. We reimplemented this GNN approach; it builds a large graph for the doc, so it's fully document-level. This baseline is very relevant as it addresses argument scattering with graphs.
- **RAAT (Liang et al., 2022)** – We include the relation-augmented transformer. We used their NAACL 2022 model code to train on our data. It's one of the latest SOTA on Chinese financial documents.
- **Text2Event (Lu et al., 2021)** – An end-to-end generative event extraction baseline. We train the generation model (with constrained decoding to format output) on our datasets. It uses BART-large as the backbone for English and a mBART for Chinese. This baseline is included to compare our cascaded framework with direct sequence-to-structure event decoding.
- **GPT-4 (Achiam et al., 2023)** – We evaluate a prompt-based in-context learning using GPT-4, to represent a powerful LLM baseline. This baseline also uses language-matched prompts: for News15, both the instruction and the demonstration example are written in Chinese, whereas for WikiEvents they are written in English. We design a prompt with a few examples (for News15, we provide one example of an extracted event; for WikiEvents, one example due to length limits). GPT-4 is asked to output events in a structured JSON. We parse its output. This baseline is zero-shot/few-shot usage of GPT-4 without fine-tuning.

This baseline selection enables a direct comparison between our cascaded design and multiple end-to-end alternatives from different paradigms. Specifically, Doc2EDAG and DE-PPN represent document-level end-to-end structured decoding, while Text2Event represents end-to-end sequence-to-structure generation. Therefore, our evaluation does not merely contrast the proposed method with earlier pipelines; it also tests whether explicit trigger fusion, role-wise span grounding, and later causal refinement are more effective than directly decoding the full event structure in one end-to-end process. For fair comparison, all non-LLM baselines are provided gold entity mentions (if the task requires entity recognition). In our setting, entities are not separately annotated (the tasks expect span extraction), so each model has to do its own span selection. We gave no additional advantage to our model in that regard.

4.5. Main results

We report results for each dataset separately in [Tables 1 and 2](#). From [Table 1](#), we see that our LLM-Agent approach achieves the best performance on the Chinese News15 dataset, with 66.3 F1 on argument extraction. This is +4.3 points higher than the strongest baseline (RAAT, 62.0 F1) and even higher (+6+ points) above others like GIT and DE-PPN. The precision of our method is notably high (69.3%), indicating that the structured approach and LLM guidance help reduce spurious extractions. Recall is also the highest (63.5%), suggesting our model finds more of the implicitly stated arguments. The GPT-4 few-shot baseline, interestingly, had the highest trigger F1 (it identifies triggers well given its understanding) but low argument recall, confirming that even GPT-4 struggles to output all arguments correctly structured. Traditional pipeline (DCFEE) has the lowest recall, as expected, and generative Text2Event lags in both precision and recall, likely due to generation errors or format mistakes.

On WikiEvents ([Table 2](#)), our model again tops the chart with 57.3 argument F1, compared to 53.7 by RAAT and 52.5 by GIT. This 3.6 F1 improvement is significant. Notably, the margin on WikiEvents is slightly smaller than in News15; this could be because WikiEvents documents often have arguments in the same sentence, so the advantage of our cross-sentence abilities is less dramatic but still present. The trigger F1 for our model is also highest (72.5%). We attribute this to the LLM's assistance in trigger detection – interestingly, GPT-4 alone didn't do well on triggers here (60.3 F1), possibly because Wikipedia language is varied and the prompt was minimal.

Table 1

Results on the News15 dataset (Chinese). Our approach achieves the highest scores; all differences $\geq 3\%$ F1 over the best baseline are statistically significant ($p < 0.05$).

Model	Trigger F1	Argument P	Argument R	Argument F1
DCFEE	68.3	57.4	45.2	50.6
Doc2EDAG	73.1	60.0	53.8	56.7
OneIE	75.4	62.1	49.3	55.0
DE-PPN	77.0	64.5	55.1	59.4
GIT	76.8	63.0	57.9	60.3
RAAT	78.5	65.2	59.0	62.0
Text2Event	69.8	58.7	50.4	54.3
GPT-4 (few-shot ICL)	81.2	55.9	47.5	51.4
Ours	83.7	69.3	63.5	66.3

Table 2

Results on WikiEvents (English). Our model outperforms prior methods, including recent transformer-based models, by a substantial margin in F1.

Model	Trigger F1	Argument P	Argument R	Argument F1
DCFEE	52.6	45.8	35.4	39.9
Doc2EDAG	59.0	50.3	41.7	45.6
OneIE	63.5	51.2	44.0	47.3
DE-PPN	66.1	53.7	46.9	50.1
GIT	67.8	55.9	49.5	52.5
RAAT	69.4	57.0	50.8	53.7
Text2Event	62.0	49.5	45.1	47.2
GPT-4 (few-shot ICL)	60.3	41.7	37.9	39.7
Ours	72.5	60.4	54.6	57.3

Comparison with End-to-End Methods. Existing literature indeed includes end-to-end event extraction models, and our experiments compare against three representative ones: Doc2EDAG, DE-PPN, and Text2Event. As shown in Tables 1 and 2, our method consistently outperforms these end-to-end baselines on both datasets. On News15, our argument F1 exceeds Doc2EDAG, DE-PPN, and Text2Event by 9.6, 6.9, and 12.0 points, respectively. On WikiEvents, the corresponding gains are 11.7, 7.2, and 10.1 points. These results suggest that, for long news documents with multiple event types and scattered arguments, directly decoding the full event structure in one end-to-end pass remains challenging, whereas our cascaded design benefits from explicit trigger fusion, role-wise span grounding, and later causal refinement. This positions our contribution more precisely: the advantage does not come merely from using a document-level architecture, but from decomposing the task into stages that better match the requirements of high-recall trigger discovery and boundary-exact argument extraction.

In both datasets, OneIE (sentence-level joint) is clearly behind document-level models, underscoring the need for document-level methods. Doc2EDAG’s performance is decent but it seems to suffer on WikiEvents, perhaps because it was designed for a very structured financial domain and struggles with open-domain Wikipedia text. RAAT and GIT are strong baselines as they incorporate a lot of document structure; our improvements over them indicate that the integration of LLM reasoning and causal links adds value beyond what those purely structural models achieve. We also measured the event-level exact match metric (how many events had all arguments correctly extracted). Our model achieved 48.7% exact event matches on News15 vs 42.0% for RAAT, and 40.5% vs 35.0% on WikiEvents – indicating a greater fraction of events fully captured.

4.6. Robustness across LLM backends

Robustness Across Proposal Backends: To examine whether the effectiveness of the framework depends on a specific LLM architecture, we keep the proposed framework unchanged and replace only the LLM used in the trigger-proposal stage with three additional representative backends, namely DeepSeek-R1, Gemini, and GLM. For clarity, these variants are reported as “Ours + backend proposal” rather than as stan-

Table 3

Robustness across different LLM backends in the trigger-proposal stage on the News15 development set. Only the proposal backend in our framework is changed; all downstream modules remain the same.

Framework variant	Trigger F1	Arg-P	Arg-R	Arg-F1
Ours + GPT-4 proposal	83.4	69.2	63.3	66.1
Ours + DeepSeek-R1 proposal	83.0	68.9	62.9	65.8
Ours + Gemini proposal	82.7	68.5	62.6	65.4
Ours + GLM proposal	82.3	68.1	62.1	65.0

Table 4

Ablation results on News15. Δ F1 is drop compared to full model.

Model Variant	Arg-P	Arg-R	Arg-F1	Δ F1
Full (Ours)	69.3	63.5	66.3	–
– LLM (no GPT prompts)	67.5	60.1	63.6	-2.7
– Agent (one-pass)	64.8	59.0	61.7	-4.6
– Causal Inference	68.1	61.0	64.3	-2.0

dalone baselines, since the rest of the framework, including trigger fusion, the QA-based argument extractor, and the causal refinement module, remains unchanged. For a controlled comparison, we evaluate these variants on the News15 development set. Table 3 reports the results.

As shown in Table 3, the proposed framework remains effective across different LLM backends. “Ours + GPT-4 proposal” achieves the best overall performance, but the alternatives remain close, with a gap of only 1.1 F1 points between the best and worst variants. This indicates that the framework is not tightly coupled to a single proprietary model. At the same time, GPT-4 still provides the strongest proposal quality, which is reflected in slightly higher trigger F1 and final argument F1. These results further support our design choice of using the LLM only as a proposal mechanism: changing the backend affects the quality of candidate event suggestions, but the downstream classifier, fusion module, and span-grounded argument extraction stage make the overall system substantially more robust than a direct end-to-end LLM baseline.

4.7. Ablation studies

To understand the contribution of each component, we perform ablation experiments on the News15 dataset. We consider three ablated variants of our model:

- **No LLM:** We remove the LLM from event detection. The system relies solely on the BERT trigger classifier. This tests the impact of LLM suggestions.
- **No Agent (One-Pass):** Instead of the agent querying each role separately, we use a one-pass argument extraction approach. Specifically, we replaced the agent with a set of role-specific taggers that label all argument spans in one sweep. This ablation evaluates the benefit of the iterative query process.
- **No Causal Inference:** We disable the causal module. Events are extracted independently with no propagation of arguments between linked events.

Table 4 shows the results of these ablations. We observe that removing any component drops performance. Without LLM prompting for triggers, the recall in particular drops (63.5→60.1). This suggests the LLM was contributing some triggers (and hence their arguments) that the classifier alone missed – perhaps triggers phrased in uncommon ways or multiple triggers in one sentence where the classifier might miss one. Precision only slightly declines without LLM (because the classifier was already quite precise), so the net F1 drop is 2.7.

Removing the Agent and doing one-pass argument extraction hurts both precision and recall. We suspect the one-pass approach often either misses arguments (no chance to focus on each role separately) or misassigns spans (the agent’s targeted questions reduce ambiguity). The

Table 5
Latency analysis of the proposed framework at inference time.

Variant	Avg. latency / document	Setting
w/o LLM trigger proposal	0.8 s	neural trigger classifier + QA agent + causal refinement
Full model	1.2 s	adds one GPT-4 trigger proposal call before fusion

agent’s benefit is the largest single factor here: a 4.6 F1 drop without it indicates that role-by-role querying yields significantly more accurate results. Turning off the causal inference mainly affects recall (63.5→61.0). This makes sense: the causal module was aimed at filling missing arguments. Precision also remains stable and is slightly higher in the full model (69.3 vs. 68.1), suggesting that the consistency check prevents most harmful false positive propagations rather than introducing a systematic precision penalty. Overall F1 drops by 2.0, confirming that causal propagation provided a modest gain by adding correct arguments that otherwise would be missed. In the development data, only about 8% of events receive a propagated filler at all, and roughly one quarter of these cases are clearly helpful. So it is a smaller contributor than the LLM proposal module or the agent, but it still provides useful complementary gains.

These ablations make the contribution of each module more explicit. The LLM-assisted trigger proposal mainly improves recall: compared with the no-LLM variant, the full model gains 3.4 recall points (63.5 vs. 60.1), indicating that the LLM helps recover missed or weakly signaled event mentions. The agent module yields the largest overall gain, improving both precision and recall by 4.5 points and resulting in a 4.6-point F1 increase over the one-pass alternative, which confirms that role-wise querying substantially reduces span omission and cross-event confusion. The causal module contributes a smaller but still meaningful improvement, mainly on recall (+2.5), showing that it helps recover otherwise missing arguments while introducing very few harmful propagations. Overall, these results validate that each proposed component contributes to the final performance, with the agent-based extraction strategy being particularly important.

4.8. Latency and computational complexity

Since our framework combines multiple components, we further analyze its inference efficiency. Let k denote the number of candidate events in a document and let $\bar{r}$ denote the average number of argument roles per event. The overall inference procedure consists of four stages: (i) one document-level LLM proposal call, (ii) one neural trigger tagging pass, (iii) $O(k\bar{r})$ role-wise QA queries, and (iv) $O(k^2)$ pairwise causal checks, followed by at most one bounded refinement pass over detected causal links. Therefore, the additional architectural cost grows linearly with the number of event-role queries and quadratically only in the number of extracted event candidates, rather than requiring a global joint search over all possible event structures. For example, News15 contains on average 1.7 events per document (with at most 4), which keeps the pairwise causal stage manageable in practice.

Table 5 shows that the full system processes a document in about 1.2 s on average, while removing the LLM trigger proposal reduces latency to 0.8 s per document. This indicates that the main additional overhead comes from the LLM-based front end, whereas the downstream neural components remain relatively lightweight. Importantly, the role-wise QA stage does not increase wall-clock time linearly with the number of roles in practice, because independent role queries are executed in parallel threads. The causal refinement stage is also expected to contribute limited runtime overhead, since it is implemented as a lightweight classifier over extracted event pairs and performs only one bounded refinement iteration for each detected causal link. Overall, these results show that our framework is suitable for offline document processing, while the no-LLM variant provides a faster accuracy-efficiency trade-off when lower latency is preferred.

Table 6

Prompt sensitivity analysis on the News15 development set. T2 is the default prompt template used in the main experiments.

Prompt template	Trigger F1	Arg-P	Arg-R	Arg-F1
T1: Minimal instruction	82.1	68.5	62.3	65.3
T2: Ontology-guided (default)	83.4	69.2	63.3	66.1
T3: Ontology + exhaustive reminder	83.7	68.7	63.5	66.0

4.9. Additional analysis

We conduct further analysis to gain insight into our model’s behavior under various conditions:

Prompt Sensitivity: Since the LLM in our framework is used only for trigger proposal rather than for direct end-to-end extraction, we further examine how much the final system depends on prompt wording. On the News15 development set, we compare three Chinese prompt templates for the LLM proposal stage: T1 uses a minimal instruction-only prompt, T2 uses an ontology-guided prompt with event-type descriptions (the default setting used in the main experiments), and T3 further adds an explicit exhaustiveness reminder and stricter output-format constraints. All downstream components, including trigger fusion, the QA-based argument extractor, and the causal refinement module, are kept unchanged. Table 6 reports the corresponding results. This analysis isolates the effect of prompt variation on the proposal stage while preserving the rest of the framework.

As shown in Table 6, the framework is somewhat sensitive to prompt design at the trigger-proposal stage, but the variation in final end-to-end performance remains limited. The minimal instruction-only template (T1) performs worst, suggesting that without ontology guidance the LLM is more likely to miss weak or ambiguous event mentions. The ontology-guided template (T2), which is used in the main experiments, achieves the best overall balance and yields the highest argument F1. Adding an explicit exhaustiveness reminder and stricter formatting constraints (T3) slightly improves trigger recall, but its final argument F1 remains very close to that of T2, indicating that the downstream classifier, fusion module, and span-grounded argument extractor moderate part of the prompt-level variation. Overall, these results support our design choice of using the LLM as a proposal mechanism rather than as the sole end-to-end extractor, since moderate changes in prompt wording do not lead to large fluctuations in final extraction quality.

Latency, Token Consumption, and Granularity–Cost Analysis: To provide a more comprehensive view of operational overhead, we further report the average token consumption of the GPT-4 trigger-proposal stage. Because the LLM is invoked only once per document and only for trigger proposal, the API-based cost of our framework is concentrated in a single document-level call, while finer extraction granularity mainly affects the number of downstream QA queries rather than repeated LLM usage. Table 7 summarizes these statistics.

In simplified form, the per-document cost can be written as

$$C_{\text{doc}} \approx C_{\text{LLM}}(D) + \sum_{i=1}^k |R(T_i)| C_{\text{QA}} + O(k^2) C_{\text{cause}} \quad (6)$$

where only the first term consumes paid API tokens. The results in Table 7 show that token usage is dominated by the single document-level GPT-4 proposal call, whereas finer extraction granularity is primarily reflected in the number of role-wise QA queries. This trade-off is controlled by three design choices: (1) the agent queries only the schema

Table 7
Token consumption and granularity-related cost statistics.

Dataset	Avg. input tokens	Avg. output tokens	Avg. total tokens	Avg. QA queries / doc
News15	472	41	513	7.3
WikiEvents	1089	58	1147	9.6

Table 8

Case study comparison between a baseline-style extraction result and our framework. The example shows that our method can distinguish related event instances and recover more complete argument structures through trigger-anchored extraction and causal refinement.

Aspect	Baseline Output	Our Framework Output
Input document	“On May 3, a fire broke out at Factory X in City Y, causing an explosion. Firefighters were dispatched to the scene; two workers were injured and sent to Z Hospital.”	
Trigger detection	Fire: “fire broke out”	Fire: “fire broke out”; Explosion: “explosion”
Argument extraction	Fire = {Date: May 3; Location: Factory X / City Y; Casualties: two workers injured}. The Explosion event is missing or incomplete.	Fire = {Date: May 3; Location: Factory X / City Y; Casualties: two workers injured}; Explosion = {Location: Factory X / City Y}.
Causal refinement	No explicit causal link between the two events is produced.	Fire → Explosion; the Cause role of Explosion is filled as “fire”.
Final result	Incomplete document-level event structure.	Two complete and causally linked event records.

Table 9
Quantitative breakdown of 50 manually inspected errors on News15.

Error source / type	Share	Typical effect
Trigger detection / fusion mistakes	20%	missed or spurious event trigger
Argument span misses / boundary errors	50%	incomplete or missing role filler
Cross-event assignment confusion	15%	argument attached to the wrong event
Language / normalization mismatch	15%	exact-match failure despite partial correctness

specific role set for each event rather than all possible ontology roles; (2) each QA query uses a local context window around the trigger together with LLM-highlighted clues instead of reprocessing the full document; and (3) the causal module performs only one bounded pass over extracted event pairs. As a result, our framework preserves fine-grained event structures while keeping both token consumption and runtime at a manageable level for offline document processing.

Effect of Document Length and Event Density: We grouped the test documents of News15 by their length (in characters) and by number of events mentioned, and evaluated our model and a baseline (RAAT) on each group. We found that for short documents (under 200 chars, typically containing 1 event), both models perform similarly (our F1 68, RAAT 65). However, for longer documents (500+ chars) with multiple events, our model maintains an F1 around 64, whereas RAAT drops to 57. This suggests our approach handles long, complex inputs more robustly. The agent’s targeted extraction likely prevents it from getting confused by multiple events, and the LLM helps isolate triggers in large text. RAAT, which relies on global attention, might diffuse focus as document length grows. Similarly, in documents with > 2 events, our model’s precision in assigning arguments to the correct event was 5% higher than RAAT’s, likely due to the agent explicitly anchoring each query to a specific trigger (avoiding cross-event confusion).

Role-Specific Performance: We examined F1 per argument role on News15. Our model achieved highest F1 on roles like Time (81%) and Location (74%), which are often explicitly stated (dates, place names) and easier to query (“when/where did it happen?”). More challenging roles were Cause (50%) and Participant (a generic role in some event types, 55%). The causal module improved the Cause role significantly – without it, Cause was only 30% F1 because many events omit the cause unless inferred. On the WikiEvents side, we saw high performance on core roles (e.g. Attacker, Target in conflict events 70% F1) but low on roles like Instrument (40%) which are rarely mentioned. Overall, our model had uniformly better per-role F1 than baselines; the gains were especially large (5–10 points) on roles that are often not in the

same sentence as the trigger (e.g., Origin of a TransferMoney event, or Beneficiary of a Transaction), indicating cross-sentence gathering is a strength.

Case Study: To make the qualitative analysis more intuitive, we supplement the original text-only case study with a comparative table. The example is a Chinese news article describing a fire and a subsequent explosion: “On May 3, a fire broke out at Factory X in City Y, causing an explosion. Firefighters were dispatched to the scene; two workers were injured and sent to Z Hospital.” This document contains two related but distinct events, namely a Fire event and an Explosion event. As shown in Table 8, baseline systems tend to extract the Fire event but either miss the Explosion event or fail to link it with the Fire event. In contrast, our framework first identifies both triggers, then uses the state-aware and trigger-anchored argument extraction agent to fill event-specific arguments, and finally applies causal refinement to infer that the Fire event causes the Explosion event. This case illustrates how the LLM-assisted trigger proposal, trigger-anchored argument extraction, and causal refinement module work together to produce more complete document-level event records.

Error Analysis: To make the remaining failure modes more explicit, we manually inspected 50 prediction errors made by our model on News15 and summarized them quantitatively in Table 9. The table shows that the dominant remaining errors come from the argument extraction stage: 50% are span misses or boundary errors, and another 15% are cross-event assignment mistakes. In other words, about 65% of the observed failures are largely attributable to the agent-side extraction stage rather than to trigger discovery. By contrast, 20% of errors originate from the front-end trigger detection and fusion stage, typically when an event mention is missed or a trigger candidate is overly general. The remaining 15% are due to language-specific normalization mismatches in the Chinese dataset, where the predicted mention is semantically correct but does not exactly match the annotated surface form. This breakdown is consistent with the ablation results and helps clarify the improvement brought by each module. The LLM-assisted trig-

Table 10
Audit of causal propagation and consistency control.

Metric	Value	Interpretation
Arg P (Full vs. w/o causal)	69.3 vs. 68.1	no precision degradation
Arg R (Full vs. w/o causal)	63.5 vs. 61.0	recall improves with causal inference
Events receiving propagated fillers	~8%	propagation is applied sparingly
Helpful propagated cases among these	~25%	only a subset materially changes the result
Sampled errors caused by harmful propagation	< 5%	false positive propagation is rare

ger proposal mainly addresses the 20% trigger-stage failures, which explains why removing it causes the largest drop in recall rather than precision. The agent module directly targets the dominant 65% of remaining failures, which is why it yields the largest overall F1 improvement in Table 4. The causal module mainly recovers missing arguments after initial extraction, so its contribution is more modest but recall-oriented. Importantly, harmful causal propagation is rare in our manual inspection: fewer than 5% of sampled error cases involve a wrong argument copied by the causal module. This suggests that the causal component is generally conservative, and that the main remaining challenge lies in boundary-exact argument extraction under multi-event and long-context conditions.

Consistency Check in Causal Propagation: To better understand the risk of error propagation, we further audit the behavior of the causal module using the development set and the sampled error cases. Table 10 summarizes the main observations. The first two rows compare the full model with the variant without causal inference, showing that the causal module improves recall from 61.0 to 63.5 while also maintaining stable precision. The remaining rows quantify how selectively propagation is actually used: only a small fraction of events receive propagated fillers, only a subset of these propagations materially helps the final output, and fewer than 5% of sampled error cases involve harmful propagation. Taken together, these observations indicate that the consistency check successfully limits false positive propagation and keeps the causal module conservative in practice.

5. Conclusions

In this work, we presented a novel document-level event extraction framework that combines the strengths of large language models, an agent-based extraction strategy, and causal reasoning. Our approach specifically addresses the challenges of multi-type heterogeneous argument extraction in long texts: it improves trigger detection with LLM knowledge, fills arguments through a state-aware querying agent that updates query formulation and context selection according to QA feedback and the current partial event record, and leverages causal relations between events to infer missing details. Extensive experiments on a Chinese news dataset and the English WikiEvents benchmark demonstrate that our model achieves state-of-the-art results, outperforming strong neural baselines by a significant margin (over 4 F1 points on average). We showed that each component – LLM augmentation, the extraction agent, and causal inference – contributes to this performance. In particular, the agent-based approach offers a flexible and effective way to handle multiple events and long documents, and integrating LLMs in a careful, complementary fashion (rather than end-to-end prompting) yields clear benefits.

Our work opens several avenues for future exploration. First, we plan to extend the framework to cross-document event extraction, where an agent could navigate multiple documents to compile event information, and causal links could be identified across documents (for example, linking an event report to its aftermath reported elsewhere). The modular nature of our system would allow an LLM to be used as an agent to decide which documents to read next, for instance. Second, we aim to incorporate temporal reasoning in addition to causal reasoning – ordering events correctly on a timeline and using temporal constraints to further validate or infer event arguments (e.g., if one event’s time is known and

it causes another, the cause must be earlier). Third, we are interested in improving the explainability of the extraction process. Our agent’s question-answer approach provides a natural opportunity to generate explanations (the questions themselves indicate why an argument was chosen). Building on techniques like back-tracing important sentences for each event, we could present users with supporting evidence for each extracted argument. Finally, given the success of combining LLMs with specialized models, we plan to investigate training a medium-sized language model (e.g., 1–3B parameters) on our task in an end-to-end manner using our system’s output as silver data – essentially distilling the LLM-agent knowledge into a single model for efficiency.

Overall, our study shows that a hybrid approach, which orchestrates the reasoning capabilities of large pre-trained models with structured task-specific modules, is a promising direction for tackling complex information extraction tasks. We hope this work inspires further research into intelligent agent strategies and the integration of causal knowledge for advancing event understanding in text.

CRedit authorship contribution statement

Bingtao Xu: Writing – review & editing, Writing – original draft, Methodology, Formal analysis, Data curation, Conceptualization; **Zongye Gu:** Resources, Project administration, Data curation; **Yanling Wang:** Writing – review & editing, Supervision, Resources, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets used in this manuscript are publicly available datasets. Detailed information about these datasets is provided in Section Dataset of this manuscript.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S. et al. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.
- Ahn, D. (2006). The stages of event extraction. In *Proceedings of the workshop on annotating and reasoning about time and events* (pp. 1–8).
- Chen, R., Qin, C., Jiang, W., & Choi, D. (2024). Is a large language model a good annotator for event extraction? In *Proceedings of the AAAI conference on artificial intelligence* (pp. 17772–17780). (vol. 38).
- Chen, Y., Xu, L., Liu, K., Zeng, D., & Zhao, J. (2015). Event extraction via dynamic multi-pooling convolutional neural networks. In *Proceedings of the 53rd Annual meeting of the association for computational linguistics and the 7th International joint conference on natural language processing (volume 1: Long papers)* (pp. 167–176).
- Cunha, L. F., Campos, R., & Jorge, A. (2023). Event extraction for portuguese: A qa-driven approach using ace-2005. In *Epia conference on artificial intelligence* (pp. 402–414). Springer.
- Du, X., & Cardie, C. (2020). Event extraction by answering (almost) natural questions. arXiv preprint arXiv:2004.13625.
- Glavaš, G., Šnajder, J., Kordjamshidi, P., Moens, M.-F., Calzolari, N., Choukri, K., Declerck, T., Loftsson, H., Maegaard, B., Mariani, J. et al. (2014). Hieve: A corpus for extracting event hierarchies from news stories. In *Proceedings of 9th Language resources and evaluation conference* (pp. 3678–3683). ELRA.

- He, Y., Hu, J., & Tang, B. (2023). Revisiting event argument extraction: Can EAE models learn better when being aware of event co-occurrences? *arXiv preprint arXiv:2306.00502*.
- Hong, Y., Zhang, J., Ma, B., Yao, J., Zhou, G., & Zhu, Q. (2011). Using cross-entity inference to improve event extraction. In *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies* (pp. 1127–1136).
- Hsu, I., Huang, K.-H., Boschee, E., Miller, S., Natarajan, P., Chang, K.-W., Peng, N. et al. (2021). Degree: A data-efficient generation-based event extraction model. *arXiv preprint arXiv:2108.12724*.
- Hsu, I., Xie, Z., Huang, K.-H., Natarajan, P., Peng, N. et al. (2023). Ampere: Amr-aware prefix for generation-based event argument extraction model. *arXiv preprint arXiv:2305.16734*.
- Li, S., Ji, H., & Han, J. (2021). Document-level event argument extraction by conditional generation. In *Proceedings of the 2021 Conference of the north american chapter of the association for computational linguistics: Human language technologies* (pp. 894–908).
- Liang, Y., Jiang, Z., Yin, D., & Ren, B. (2022). Raat: Relation-augmented attention transformer for relation modeling in document-level event extraction. *arXiv preprint arXiv:2206.03377*.
- Lin, J., & Chen, Q. (2021). Poke: A prompt-based knowledge eliciting approach for event argument extraction. *arXiv preprint arXiv:2109.05190*.
- Lin, Y., Ji, H., Huang, F., & Wu, L. (2020). A joint neural model for information extraction with global features. In *Proceedings of the 58th Annual meeting of the association for computational linguistics* (pp. 7999–8009).
- Liu, J., Zhang, Z., Guo, Z., Sun, X., Jin, L., Li, X. et al. (2023). Event causality extraction via implicit cause-effect interactions. In *The 2023 Conference on empirical methods in natural language processing*.
- Liu, X., Luo, Z., & Huang, H. (2018). Jointly multiple events extraction via attention-based graph information aggregation. *arXiv preprint arXiv:1809.09078*.
- Lu, Y., Lin, H., Xu, J., Han, X., Tang, J., Li, A., Sun, L., Liao, M., & Chen, S. (2021). Text2event: Controllable sequence-to-structure generation for end-to-end event extraction. *arXiv preprint arXiv:2106.09232*.
- Ma, Y., Wang, Z., Cao, Y., Li, M., Chen, M., Wang, K., & Shao, J. (2022). Prompt for extraction? PAIE: Prompting argument interaction for event argument extraction. *arXiv preprint arXiv:2202.12109*.
- Nguyen, T., & Grishman, R. (2018). Graph convolutional networks with argument-aware pooling for event detection. In *Proceedings of the AAAI conference on artificial intelligence*. (vol. 32).
- Nguyen, T. H., Cho, K., & Grishman, R. (2016). Joint event extraction via recurrent neural networks. In *Proceedings of the 2016 Conference of the North American chapter of the association for computational linguistics: human language technologies* (pp. 300–309).
- Nguyen, T. H., & Grishman, R. (2015). Event detection and domain adaptation with convolutional neural networks. In *Proceedings of the 53rd Annual meeting of the association for computational linguistics and the 7th international joint conference on natural language processing (volume 2: Short papers)* (pp. 365–371).
- Nguyen, T. M., & Nguyen, T. H. (2019). One for all: Neural joint modeling of entities and events. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 6851–6858). (vol. 33).
- Pan, B., Li, Y., Wang, S., Li, X., Li, D., Liao, J., & Zheng, J. (2024). Document-level event extraction via information interaction based on event relation and argument correlation. In *Proceedings of the 2024 Joint international conference on computational linguistics, language resources and evaluation (LREC-COLING 2024)* (pp. 5156–5166).
- Paolini, G., Athiwaratkun, B., Krone, J., Ma, J., Achille, A., Anubhai, R., Santos, C. N. d., Xiang, B., & Soatto, S. (2021). Structured prediction as translation between augmented natural languages. *arXiv preprint arXiv:2101.05779*.
- Sundheim, B. M. (1991). Overview of the third message understanding evaluation and conference. In *Third message understanding conference (MUC-3): Proceedings of a conference held in san diego, california, may 21–23, 1991*.
- Wadden, D., Wennberg, U., Luan, Y., & Hajishirzi, H. (2019). Entity, relation, and event extraction with contextualized span representations. *arXiv preprint arXiv:1909.03546*.
- Wang, X., Gui, L., & He, Y. (2023a). Document-level multi-event extraction with event proxy nodes and hausdorff distance minimization. *arXiv preprint arXiv:2305.18926*.
- Wang, X., Zhou, W., Zu, C., Xia, H., Chen, T., Zhang, Y., Zheng, R., Ye, J., Zhang, Q., Gui, T. et al. (2023b). Instructua: Multi-task instruction tuning for unified information extraction. *arXiv preprint arXiv:2304.08085*.
- Xu, R., Liu, T., Li, L., & Chang, B. (2021). Document-level event extraction via heterogeneous graph-based interaction model with a tracker. *arXiv preprint arXiv:2105.14924*.
- Yang, H., Chen, Y., Liu, K., Xiao, Y., & Zhao, J. (2018). Dcfee: A document-level chinese financial event extraction system based on automatically labeled training data. In *Proceedings of ACL 2018, system demonstrations* (pp. 50–55).
- Yang, H., Sui, D., Chen, Y., Liu, K., Zhao, J., & Wang, T. (2021). Document-level event extraction via parallel prediction networks. In *Proceedings of the 59th Annual meeting of the association for computational linguistics and the 11th International joint conference on natural language processing (volume 1: Long papers)* (pp. 6298–6308).
- Yu, Y., Wang, Y., Ma, Y., Li, J., Lu, K., Huang, Z., & Chua, T. S. (2024). EE-LCE: An event extraction framework based on llm-generated cot explanation. In *International conference on knowledge science, engineering and management* (pp. 28–40). Springer.
- Zhao, W., Zhao, Y., Jiang, X., He, T., Liu, F., & Li, N. (2021). Efficient multiple biomedical events extraction via reinforcement learning. *Bioinformatics*, 37 (13), 1891–1899.
- Zheng, S., Cao, W., Xu, W., & Bian, J. (2019). Doc2EDAG: An end-to-end document-level framework for chinese financial event extraction. *arXiv preprint arXiv:1904.07535*.
- Zhu, T., Qu, X., Chen, W., Wang, Z., Huai, B., Yuan, N. J., & Zhang, M. (2022). Efficient document-level event extraction via pseudo-trigger-aware pruned complete graph. In: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*, (pp. 4552–4558).